Relazione Progetto

Giannuzzi Riccardo

Matricola: 7199750

https://github.com/Riccardo-Giannuzzi/shop-manager

Automated Software Testing

Università degli Studi di Firenze

Master's degree in Software: Science and Technology

Anno Accademico 2025/2026



Indice

1 Descrizione caso di studio **1**
1.1 Modello di Dominio 1

2 Scelte di Design e di Implementatazione **2**
2.1 Model 2
2.2 Controller 2
2.3 Repository 3
2.4 View 3

3 Tecnologie High-Rating **5**
3.1 BDD with Cucumber 5
3.2 Database relazionale (MariaDB) 6

4 Tecnologie e Frameworks **7**

5 Compilazione ed Esecuzione **8**

1 Descrizione caso di studio

Il progetto implementa la gestione delle vendite di un ipotetico negozio in cui un acquirente può acquistare un prodotto una sola volta. Un possibile contesto reale di riferimento in cui gli acquisti sono gestiti in questo modo è la vendita di beni digitali come le licenze di un software oppure l'acquisto di un libro digitale. Questo significa che sono possibili due soli casi: il cliente ha acquistato un prodotto oppure non ha effettuato alcun acquisto. Quindi non è necessario gestire acquisti multipli.
Il caso d'uso che questa applicazione mira a risolvere è quello di un dipendente di un negozio che registra i clienti, gestisce il catalogo dei prodotti e tiene traccia delle vendite effettuate. Pertanto lo scope del progetto prevede l'implementazione delle operazioni fondamentali relative alla gestione delle entità del sistema ovvero la visualizzazione, l'aggiunta e l'eliminazione dei prodotti, dei clienti e degli acquisti tramite un'interfaccia grafica interattiva.
Per quanto riguarda l'eliminazione non è ammesso eliminare un prodotto o un cliente se sono coinvolti in un acquisto. Questo significa che per eliminare un prodotto che è stato acquistato almeno una volta prima è necessario eliminare tutti gli acquisti ad esso legati.
Queste scelte mantengono un livello di complessità del modello e della logica dell'applicazione adeguato per potersi concentrare sugli aspetti più rilevanti del corso.

1.1 Modello di Dominio

Il modello di dominio dell'applicazione è costituito dalle entità Customer e Product, collegate attraverso la relazione Purchase. Tale relazione rappresenta un'associazione molti-a-molti: un cliente può acquistare zero o più prodotti e, allo stesso modo, un prodotto può essere acquistato da zero o più clienti.

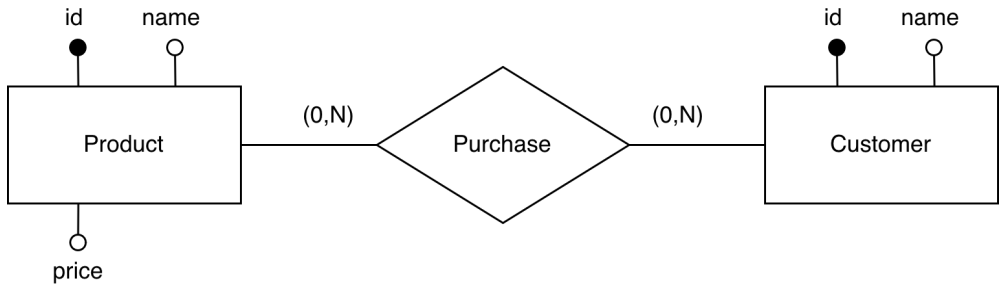


Figura 1: Schema ER del Modello di Dominio

Come mostrato in Figura 1, l'entità Product è caratterizzata da un identificativo univoco, da un nome e da un prezzo mentre l'entità Customer da un identificativo univoco e un nome. La relazione Purchase associa un cliente a un prodotto e viene quindi identificata dalla coppia formata dall'identificativo del cliente e dall'identificativo del prodotto. In questo modo, per una determinata coppia cliente-prodotto può esistere al più un acquisto.

2 Scelte di Design e di Implementatazione

Dal punto di vista architetturale, il progetto è stato organizzato secondo il pattern Model-View-Controller (MVC), con l'obiettivo di separare chiaramente le diverse responsabilità dell'applicazione. Il Model rappresenta i dati e ne gestisce la persistenza attraverso i repository, la View si occupa dell'interfaccia grafica e dell'interazione con l'utente e il Controller contiene la logica applicativa e coordina le operazioni tra View e Model. Qusta separazione permette di realizzare un'applicazione modulare e che quindi rispetti i principi SOLID per lo sviluppo di codice pulito, flessibile e testabile. Inoltre l'utilizzo del Test Driven Development (TDD), per la sua natura impelmentativa, ha controbuito a realizzare un'applicazione che segua tali principi. Questo favorisce la realizzazione di classi con responsabilità limitate, dipendenze esplicite e componenti facilmente isolabili durante il testing.

2.1 Model

Il Model rappresenta le entità principali del dominio applicativo ed è composto dalle classi Customer, Product e Purchase che vengono utilizzate per rappresentare e trasportare i dati tra i diversi componenti. Tali classi sono state escluse dal coverage perchè al loro interno non vi è logica effettiva se non setter, getter, equals, hashCode e toString generati automaticamente con gli helper di Eclipse.

2.2 Controller

Sono stati definiti tre controller, uno per ciascuna delle principali entità del dominio. Ognuno di essi ha la responsabilità di gestire le operazioni relative soltanto alla propria entità: visualizzazione dei dati esistenti, l'inserimento di nuovi elementi e la loro eliminazione.
I controller dipendono dalle interfacce dei repository e le intrerfacce delle view, con le relative implementazioni concrete che vengono fornite a runtime tramite il costruttore. In questo modo, secondo il principio di Dependency Inversion, i controller restano indipendenti dall'effettiva tecnologia utilizzata per la persistenza dei dati o dalla specifica implementazione dell'interfaccia grafica. Quindi i controller sono il punto cardine in cui viene effettivamente implementata la parte logica dell'applicazione.
Dato che il dominio stabilisce che non è possibile eliminare un prodotto o un acquirente se è coinvolto in un acquisto, prima di effettuare l'operazione di rimozione, il controller deve verificare attraverso il repository degli acquisti che non esistano relazioni associate all'entità. Per questo motivo i controller devono accedere ai dati gestiti da repository diversi, tuttavia le interfacce dei repository sono state mantenute separate, in modo da rispettare l'Interface Segregation Principle così ogni controller dipende soltando dai metodi che usa effettivamente e l'applicazione resta flessibile in caso di eventuali futuri cambiamenti nei requisiti.
Il testing dei controller è stato effettuato in isolamento tramite Mockito. Le dipendenze verso repository e view sono state sostituite con mock, permettendo di verificare sia la

logica implementata dai controller sia le corrette interazioni con i componenti dipendenti senza coinvolgere database o interfacce grafiche reali.

2.3 Repository

Per gestire l'accesso ai dati è stato adottato il Repository Pattern, con l'obiettivo di separare la logica applicativa dalle tecnologie di persistenza utilizzate. Per ciascuna entità del dominio è stata quindi definita un'interfaccia repository, mentre le operazioni concrete di accesso ai dati sono state implementate separatamente per MongoDB e MariaDB. Le implementazioni per MongoDB utilizzano il driver ufficiale, mentre quelle per MariaDB si basano su JDBC (Java Database Connectivity), l'API standard di Java per l'accesso ai database relazionali. Per il testing dei repository non sono stati utilizzati mock delle API dei database, in quanto si tratta di dipendenze esterne. Per MongoDB è stato usato il server in-memory mentre per MariaDB sono stati usati container reali tramite Testcontainers.

2.4 View

La gestione dei Customer e delle Purchase è stata raggruppata all'interno della stessa schermata, Figura 3, in modo da rendere più immediata l'interazione tra clienti e acquisti. Selezionando un cliente, l'interfaccia mostra separatamente i prodotti che può ancora acquistare e gli acquisti già effettuati. Questa organizzazione sfrutta direttamente il vincolo del dominio secondo cui un cliente può acquistare uno stesso prodotto al massimo una volta.

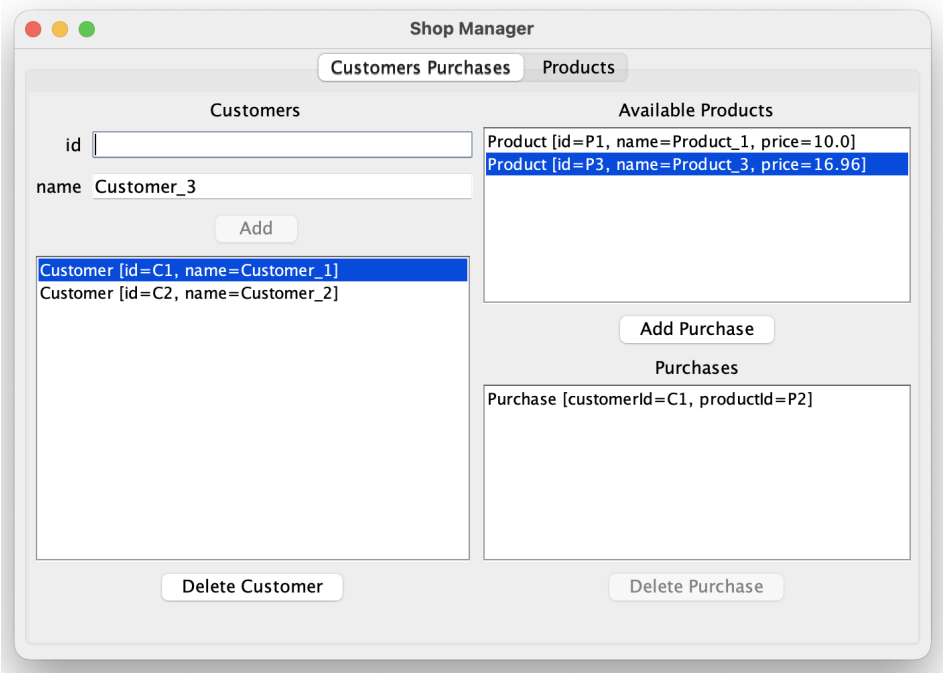


Figura 2: Interfaccia grafica per la gestione dei Customer e dei Purchase

I prodotti ancora disponibili e quelli già acquistati vengono quindi visualizzati in liste distinte, rendendo intuitive le operazioni di aggiunta e rimozione di un acquisto tramite selezione dell'elemento e pressione del relativo pulsante.
La gestione dei product è invece collocata in una schermata separata, mostrata in Figura 3, dal momento che, nel caso d'uso considerato, la modifica del catalogo dei prodotti rappresenta un'operazione meno frequente rispetto alla consultazione dei clienti e alla registrazione degli acquisti.

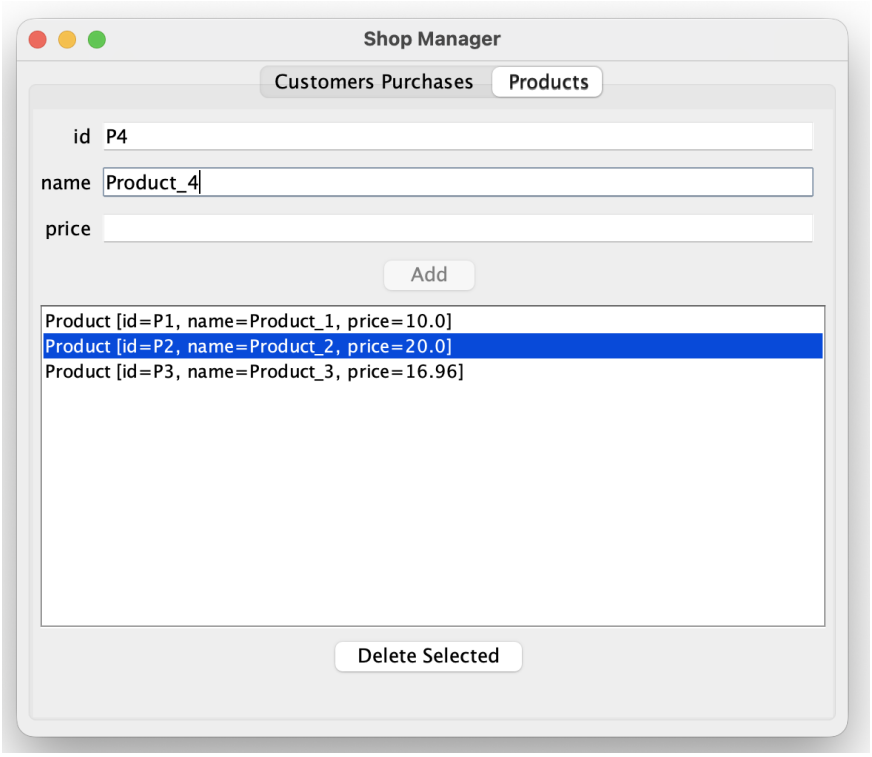


Figura 3: Interfaccia grafica per la gestione dei Product

Per evitare la presenza di più finestre indipendenti, l'interfaccia è stata organizzata all'interno di una finestra principale contenente un componente JTabbedPane. Le due schermate principali vengono quindi presentate come tab separate, in modo analogo alle schede di un browser. Particolare attenzione è stata posta nel mantenere consistente lo stato della View durante il passaggio da una tab all'altra. Al cambio di schermata, i dati delle liste visualizzate vengono aggiornati in modo da riflettere lo stato corrente dei repository ed evitare la presenza di informazioni non più valide o non sincronizzate.
La GUI è stata implementata con Java Swing, uitlizzando il plugin WindowBuilder di Eclipse e testata in isolamento usando AssertJ Swing e Mockito per mockare i controller.

3 Tecnologie High-Rating

3.1 BDD with Cucumber

Come nuova tecnologia è stato utilizzato Cucumber per introdurre test basati sul paradigma Behavior Driven Development (BDD). Tramite questo framework è possibile descrivere il comportamento atteso dell'applicazione attraverso scenari leggibili anche a un livello più vicino ai requisiti funzionali.
Uno scenario rappresenta un esempio concreto del comportamento atteso dell'applicazione e può essere considerato equivalente a un caso di test. Ogni scenario è composto da una sequenza di step organizzati secondo la struttura Given-When-Then: i passi Given descrivono il contesto iniziale e le precondizioni del test, i passi When rappresentano l'evento o l'azione eseguita dall'utente, mentre i passi Then specificano il risultato atteso e quindi la condizione da verificare. Ulteriori step possono essere espressi tramite keyword And e But. Nel progetto sono stati definiti scenari a differenti livelli di astrazione. Un esempio di scenario ad alto livello è quello relativo all'aggiunta di un acquisto:

```
Scenario: Add a purchase
  Given The user selects a customer from the list
  And The user selects a product from the available products
  When The user clicks the "Add Purchase" button
  Then The purchase list contains the new purchase
```

Figura 4: Scenario high-level per l'aggiunta di un acquisto

Altri scenari utilizzano invece step più espliciti, nei quali vengono indicati i dati presenti nel database, gli elementi selezionati nell'interfaccia e i pulsanti premuti. Un esempio di tali scenari è quello relativo al tentativo di eliminazione di un cliente coinvolto in un acquisto:

```
Scenario: Delete a customer that has a purchase
  Given The database contains the customers with the following values
    | C1 | customer1 |
  And The database contains the products with the following values
    | P1 | product1 | 10.0 |
  And The database contains the purchases with the following values
    | C1 | P1 |
  And The Customer View is shown
  When The user selects the customer with id "C1"
  And The user clicks the "Delete Customer" button
  Then An error is shown containing the following values
    | C1 | customer1 |
```

Figura 5: Scenario dettagliato per il tentativo di eliminazione di un cliente con acquisti

Questi scenari sono stati scritti utilizzando il linguaggio Gherkin e organizzati all'interno della directory `src/bdd/resources`, mentre le relative definizioni degli step in Java sono state collocate in `src/bdd/java`.
Ogni frase Gherkin è associata a una step definition Java tramite le annotazioni messe a disposizione da Cucumber, come `@Given`, `@When` e `@Then`. Le step definition relative all'interfaccia grafica utilizzano AssertJ Swing per individuare e manipolare i componenti della finestra, mentre quelle relative alla preparazione dello stato iniziale accedono direttamente al database di test.
Poiché questi scenari verficano il funzionamento l'applicazione nella sua interezza è necessario che durante la loro esecuzione sia disponibile un'istanza MongoDB reale. In questo caso non è stato utilizzato Testcontainers direttamente dagli step Cucumber questo perchè il codice dei test è in un file differente rispetto a quello in cui vengono effettivamente eseguiti. Il database viene invece avviato tramite il Fabric8 Docker Maven Plugin prima della fase di integration test e arrestato automaticamente al termine. In questo modo l'esecuzione degli scenari BDD rimane integrata nel normale ciclo di vita Maven e non è richiesto l'avvio manuale del database.
Rispetto a normali test end-to-end scritti direttamente in Java, l'utilizzo di Cucumber offre una maggiore leggibilità delle specifiche e la riusabilità delle step definition. Infatti una volta implementato uno step, questo può essere utilizzato in più scenari e anche in feature differenti, purché la frase Gherkin corrisponda alla stessa definizione. In questo modo è possibile comporre nuovi scenari riutilizzando operazioni già implementate, riducendo la duplicazione del codice di test.

3.2 Database relazionale (MariaDB)

Oltre all'implementazione basata su MongoDB, il progetto supporta anche un database relazionale SQL tramite MariaDB. L'applicazione permette di specificare facilmente la scelta del database che viene effettuata a runtime tramite i parametri da riga di comando definiti con Picocli. Il parametro `--db-type` permette di specificare se utilizzare MongoDB oppure MariaDB. In base ai parametri specificati l'applicazione istanzia le implementazioni concrete dei repository in base al database selezionato e le passa ai controller.
Il modello relazionale è stato definito in modo da riflettere i vincoli del caso di studio. In particolare, la relazione Purchase è rappresentata da una tabella con chiave primaria composta da customer_id e product_id, garantendo che una stessa coppia cliente-prodotto possa comparire al massimo una volta.
L'implementazione MariaDB utilizza JDBC per interagire con il database relazionale e le operazioni di lettura, inserimento ed eliminazione vengono eseguite tramite PreparedStatement, mentre i risultati delle query vengono convertiti nelle corrispondenti entità del dominio. Le eccezioni specifiche del database vengono infine incapsulate in una RepositoryException, evitando di propagare ai controller dettagli legati all'implementazione. Questo mantiene l'interfaccia dei repository consistente indipendentemente dall'implementazione concreta sottostante.

4 Tecnologie e Frameworks

Di seguito viene riportata una selezione delle principali tecnologie e dei framework utilizzati per la realizzazione del progetto, accompagnata da una breve descrizione del loro ruolo e delle modalità con cui sono state applicate.

Maven
Utilizzato come strumento principale per la gestione del ciclo di build del progetto, delle dipendenze e dei plugin. Tramite Maven vengono eseguiti i test, generati i report di coverage e mutation testing e costruito il file JAR eseguibile dell'applicazione.

JUnit4
Utilizzato come framework principale per la scrittura e l'esecuzione dei test insieme ad AssertJ e AssertJ Swing, quest'ultimo usato per testare la GUI Swing.

Mockito
Utilizzato nei test per creare mock delle dipendenze e isolare le classi sottoposte a test. In particolare è stato impiegato per verificare le interazioni tra controller, repository e view senza coinvolgere implementazioni reali delle dipendenze.

Jacoco
Utilizzato per verificare il raggiungimento del 100% di code coverage e di branch coverage da parte dalla suite di test. Il report generato dal plugin viene inoltre utilizzato da Coveralls e SonarCloud.

PIT
Utilizzato per il mutation testing della suite di test. Nel progetto è stato configurato con i mutatori `STRONGER` e con una mutation threshold del 100%. Per evitare di rallentare eccessivamente l'esecuzione l'analisi è stata limitata alle parti considerate più significative della logica applicativa ovvero i package contenenti i controller e i repository.

Github Actions
Utilizzato per implementare la pipeline di Continuous Integration del progetto. Ad ogni push o pull request vengono eseguite automaticamente le principali fasi di build e test, insieme alla generazione della coverage e alle analisi di qualità previste dal progetto.

SonarCloud
Utilizzato per l'analisi statica e il monitoraggio della qualità del codice. SonarCloud viene eseguito all'interno della pipeline di Continuous Integration per conrtollare coverage, maintainability, reliability, security e technical debt con l'obbiettivo di mantenere un technical debt pari a 0. Le uniche esclusioni configurate nel file pom.xml sono state per le asserzioni non riconosciute di AssertJ Swing e le naming conventions per le variabili generate da WindowBuilder.

Docker & docker-compose
Utilizzati per creare ambienti riproducibili per l'esecuzione dei database MongoDB e MariaDB.

Testcontainers
Utilizzato nei test di integrazione per avviare automaticamente istanze reali di MongoDB e MariaDB all'interno di container Docker senza dipendere da database configurati manualmente sulla macchina dello sviluppatore.

Fabric8
Utilizzato per integrare l'avvio e l'arresto di un container MongoDB direttamente nel ciclo di vita Maven. In particolare viene impiegato per fornire il database necessario all'esecuzione degli scenari BDD durante le fasi di ent to end test.

Picocli
Utilizzato per la gestione dei parametri da riga di comando dell'applicazione. In questo modo è possibile scegliere facilmente quale implementazione del database utilizzare, MongoDB oppure MariaDB, e di specificare i relativi parametri di connessione.

5 Compilazione ed Esecuzione

Purché siano soddisfatti i seguenti requisiti, la compilazione e l'esecuzione del progetto dovrebbero essere abbastanza semplici:

- Java 17 o successivo
- Maven
- Docker e Docker Compose
- Make

Il Makefile fornisce soluzioni one-command per eseguire i test e la coverage, il mutation testing e l'applicazione finale utilizzando MongoDB oppure MariaDB.
Per eseguire l'intera suite di test e generare il report di coverage JaCoCo:

```
make run-coverage
```

Per eseguire il mutation testing con PIT:

```
make run-pit
```

Per compilare ed eseguire l'applicazione utilizzando MongoDB:

```
make run-app-mongo
```

Per compilare ed eseguire l'applicazione utilizzando MariaDB:

```
make run-app-maria
```

Questi ultimi due comandi compilano l'applicazione, avviano il container relativo al database selezionato, attendono che il servizio sia disponibile e infine lanciano l'applicazione.
Il repository include le configurazioni Docker Compose richiseste sia per il database MongoDB che per MariaDB.

Al termine dell'esecuzione, i container dei database possono essere arrestati e rimossi tramite:

```
make stop-containers
```

In alternativa, qualora non fosse disponibile il comando Make, è possibile eseguire manualmente i corrispondenti comandi Maven e Docker Compose contenuti nel Makefile.